

```
string path = @"C:\Users\posera\myfile.txt";
if (File.Exists(path))
{
    using (StreamWriter sw = File.CreateText(path))
    {
        sw.WriteLine("Hello!");
    }
    using (StreamReader sw = File.OpenText(path))
    {
        string line = "";
        while ((line = sw.ReadLine()) != null)
        {
            Console.WriteLine(line);
        }
    }
    string text = File.ReadAllText(path);
    Console.WriteLine(text);
}
}
```

You can also create StreamReaders and StreamWriters over files directly via their constructors. Note that other operations may produce streams, e.g., network connections that you will use in a similar manner. In cases like a network connection where reducing system calls are important, *BufferedStream* is an available wrapper class to buffer read and write calls.

5.4.2 Deterministic finalisation and the *IDisposable* interface

The using block above is an example of deterministic finalisation. Recall that Java provides a notion of a finaliser for an object. The finaliser is theoretically responsible for cleaning up resources (e.g., fonts, files, network sockets) that a class utilises. However, the finaliser has no strong guarantees about when it will be called, just that it will be called sometime after the object is eligible to be garbage collected. This may work in some scenarios, but if you need to release that file or network socket immediately, you can't rely on a finaliser.

C# has a notion of finaliser, too, borrowing the syntax of C++ destructors. If that you are familiar with destructors, finalisers have very different semantics.

```
public class ExampleFinalizer
{
```